



Estudiantes:

Caicho Almeida Shaney Odessa
Cedeño Pinto Scarlett Yamilett

Curso:

CDDEIA-ELNO-4-2

Asignatura:

Procesamiento de Lenguaje Natural

Docente:

Msc. Ángel Cuenca Ortega

Tema:

Agente Conversacional para Admisión y Nivelación de la UG

1. Índice

1. Índice	2
2. Introducción	3
3. Diseño de Intenciones y Entidades	4
Intenciones	4
Entidades	4
Corpus de Información Académica.....	5
4. Descripción de la Arquitectura del Agente.....	5
5. Justificación de Decisiones de Diseño y Técnicas Adoptadas.....	6
TF – IDF en lugar de Bag Of Words.....	7
Similitud coseno en lugar de distancia euclidiana	7
Preprocesamiento antes de vectorizar	7
Umbral de confianza y respuesta de Fallback.....	8
Almacenamiento en JSON separado del código	8
Manejo de entradas problemáticas con try-except	9
6. Resultados de la Evaluación	9
Limitaciones observadas	10
7. Conclusiones	11
8. Referencias.....	12

2. Introducción

Cada periodo académico, el proceso de admisión y nivelación de la Universidad de Guayaquil (UG) genera un alto volumen de consultas repetitivas por parte de los aspirantes: requisitos de inscripción, fechas de registro, creación de cuentas, aprobación de asignaturas, cupos y aceptación de carreras. Gran parte de esa información se encuentra dispersa entre el portal oficial de admisiones, la página de nivelación y las redes sociales institucionales, lo que dificulta su consulta ágil.

Este proyecto aplica técnicas clásicas de PLN para construir un agente conversacional que responda en español, preguntas frecuentes sobre admisión y nivelación de la UG a partir de información extraída de fuentes oficiales. El agente se centra en la detección de intenciones y la recuperación de respuestas predefinidas, y además incorpora un corpus de información académica para enriquecer sus respuestas con datos actualizados de facultades y carreras.

La solución no requiere despliegue en producción ni utiliza modelos de Deep learning o Transformers, el enfoque es exclusivamente PLN clásico ejecutable de forma local o en Google Colab, con una interfaz web construida con Gradio.

3. Diseño de Intenciones y Entidades

Intenciones

Se definieron 22 intenciones que cubren el flujo completo del proceso de admisión y nivelación de la UG, cada una tiene asociado un conjunto de utterances (frases representativas de como un usuario puede formularla) y una o varias respuestas predefinidas almacenadas en el archivo intenciones.json. Cada intención cuenta con entre 14 y 31 utterances con variedad de longitud (palabras sueltas, frases cortas y frases largas), errores tipográficos intencionales y sinónimos, lo que permite que el modelo TF-IDF generalice correctamente sin depender de coincidencias exactas. Las intenciones se organizan en tres grupos:

Grupo	Intenciones
Gestión general	saludo, despedida, fuera_dominio
Proceso de admisión	proceso_admision, requisitos_admision, registro_nacional, fechas_registro, aplicacion_evaluaciones, asignacion_cupo, aceptacion_cupo, segunda_opcion_carrera
Nivelación y plataformas	crear_cuenta_siug, plataforma_nivelacion, matricula_nivelacion, aprobacion_nivelacion, clases_nivelacion, directrices_nivelacion, proceso_retiro_nivelacion
Información académica	oferta_academica, informacion_carrera, carreras_disponibles, costo_nivelacion, contacto_admision, problema_tecnico

Entidades

El agente detecta entidades relevantes del dominio mediante comparación de cadenas y expresiones regulares, aplicadas sobre el texto original (sin preprocesar) para preservar información contextual. Cuando se detecta una carrera, el agente personaliza la respuesta incluyendo el nombre de la carrera identificada, lo que mejora la experiencia del usuario al demostrar que el sistema comprendió el contexto específico de la consulta. Las entidades reconocidas son:

- **Carrera:** nombres de carreras ofertadas por la UG, se detectan comparando el texto del usuario contra una lista definida en el código.
- **Término del proceso:** términos específicos del flujo de admisión como "cupos aceptados", "registro nacional", "siug", "matricula", "evaluación de conocimientos". Se detectan comparando contra la lista TERMINOS_PROCESO.
- **Cedula:** números de 10 dígitos consecutivos útiles para identificar cuando el usuario consulta sobre fechas de registro personalizadas.

Corpus de Información Académica

Adicionalmente se integra un corpus_nivelacion.json con 10 secciones de información estructurada extraída de fuentes oficiales, este incluye datos sobre la oferta académica (59 carreras en 17 facultades), información detallada de cada carrera (facultad, título, semestres, modalidad), el proceso de admisión, la asignación de cupos, la matrícula de nivelación, la aprobación del curso y el proceso de retiro. El agente usa similitud coseno sobre TF-IDF para recuperar el fragmento más relevante del corpus y enriquecer la respuesta predefinida con información adicional cuando corresponde.

4. Descripción de la Arquitectura del Agente

La separación entre datos (JSON) y lógica (Python) permite que cualquier miembro del equipo actualice las intenciones sin modificar el código del agente, la función `responder_con_corpus` combina la respuesta predefinida de la intención detectada con el fragmento más relevante del corpus, aplicando un boost de similitud cuando el tag del corpus coincide con la intención detectada (`similitudes[i] *= 1.5`). La interfaz de usuario se construye con Gradio ChatInterface, que gestiona automáticamente el historial de la conversación y muestra, junto a cada respuesta, la intención detectada y el valor de similitud para que el usuario pueda verificar el comportamiento del agente. El agente sigue una arquitectura de pipeline en 6 etapas que se ejecutan en secuencia para cada consulta del usuario:

Etapas	Función	Implementación en el código
1. Carga de datos	Leer intenciones.json y corpus_nivelacion.json desde GitHub	<code>cargar_datos(url),</code> <code>cargar_corpus(url_corpus)</code>
2. Preprocesamiento	Minúsculas, quitar tildes, puntuación, stopwords, stemming	<code>preprocesar(texto)</code>
3. Vectorización TF-IDF	Convertir utterances y corpus en vectores numéricos	<code>entrenar(utterances),</code> <code>TfidfVectorizer(ngram_range=(1,2))</code>
4. Detección de intención	Similitud coseno entre consulta y utterances	<code>detectar_intencion(consulta, ...)</code>
5. Extracción de entidades	Detectar carrera, término, cédula	<code>extraer_entidades(consulta)</code>
6. Respuesta y fallback	Umbral 0.25, enriquecer con corpus, manejo try-except	<code>responder_con_corpus(...),</code> <code>responder_umbral(...)</code>

5. Justificación de Decisiones de Diseño y Técnicas Adoptadas

TF – IDF en lugar de Bag Of Words

Se usó TfidfVectorizer de scikit-learn con ngram_range=(1,2) y sublinear_tf=True; el factor IDF (Inverse Document Frequency) penaliza términos que aparecen en muchas intenciones, como "ug" o "como", y premia los que son discriminativos de una sola intención, como "zoom" (exclusivo de clases_nivelacion) o "cedula" (exclusivo de fechas_registro). Con conteos simples, esas palabras frecuentes pero no informativas dominarían los vectores y reducen la precisión. El parámetro sublinear_tf aplica escala logarítmica $1+\log(\text{tf})$ para suavizar el efecto de palabras muy repetidas dentro de un mismo utterance.

```
vectorizador = TfidfVectorizer(ngram_range=(1, 2), sublinear_tf=True)
matriz = vectorizador.fit_transform(utterances)
```

Similitud coseno en lugar de distancia euclidiana

La similitud coseno mide el ángulo entre dos vectores, lo que la hace independiente de su magnitud (longitud), por ejemplo si un usuario escribe "matricula" (1 token) y otro escribe "como hago la matricula en la ug" (6 tokens), ambos producen vectores de diferente norma pero apuntan en la misma dirección semántica. La distancia euclidiana seria grande entre ambos por diferencia de magnitud, aunque sean semánticamente idénticos. Con coseno ambos obtienen similitud alta porque el ángulo entre sus vectores es pequeño.

```
similitudes = cosine_similarity(vector_consulta, matriz)[0]
idx_mejor = similitudes.argmax()
```

Preprocesamiento antes de vectorizar

Se aplica una cadena de transformaciones antes de entrenar el TF-IDF y antes de

vectorizar cada consulta: conversión a minúsculas, eliminación de tildes mediante normalización Unicode NFKD, eliminación de puntuación, eliminación de stopwords en español (con NLTK) y stemming con SnowballStemmer; este último paso es crítico para el español: sin stemming, "matriculo", "matricula", "matricularse" y "matriculación" son cuatro tokens distintos con vectores ortogonales. Con stemming todos colapsan a la raíz "matricul", lo que permite detectar la intención correcta independientemente de la conjugación o forma gramatical usada por el usuario.

```
texto = unicodedata.normalize("NFKD", texto).encode("ascii",
"ignore").decode("ascii")
tokens = [stemmer_es.stem(t) for t in tokens if t not in STOPWORDS_ES]
```

Umbral de confianza y respuesta de Fallback

Se estableció un umbral de confianza de 0.25 sobre la similitud coseno, fue elegido empíricamente ya que un umbral demasiado alto (ej. 0.5) rechazaría consultas validas con vocabulario no visto en entrenamiento; umbral demasiado bajo (ej. 0.05) aceptaría consultas completamente fuera del dominio. Con 0.25 el agente acepta variaciones lingüísticas normales y errores tipográficos moderados, y rechaza preguntas sin relación con la admisión de la UG (como "quien gana el mundial" cuya similitud coseno con cualquier utterance del dominio es prácticamente 0). Sin umbral, ese tipo de consultas recibirían una respuesta incorrecta, lo que dañaría la credibilidad del agente.

Almacenamiento en JSON separado del código

Las intenciones, utterances y respuestas se almacenan en intenciones. json y el corpus en corpus_nivelacion.json, ambos alojados en GitHub y cargados dinámicamente al inicio del programa, esta separación respeta el principio de separación de responsabilidades: si se deben agregar nuevas intenciones o actualizar respuestas por cambios en el proceso de admisión de la UG, basta con editar el archivo JSON sin tocar una sola línea del código Python. Además,

el JSON puede versionarse y auditarse independientemente, lo que facilita la colaboración entre integrantes del equipo.

```
url = "https://raw.githubusercontent.com/.../intenciones.json"
intenciones_datos = pd.read_json(url)
```

Manejo de entradas problemáticas con try-except

La función `responder_umbral` y `responder_con_corpus` envuelven toda la lógica de detección y respuesta en un bloque try-except, garantizando que el agente nunca se detenga por una entrada vacía, un None, o texto que cause errores en el vectorizador (como una cadena con solo signos de puntuación que queda vacía tras el preprocesamiento). En cualquier caso de excepción, el bloque except captura el error y devuelve una respuesta de fallback estándar sin interrumpir la sesión del usuario. Adicionalmente se valida explícitamente que la consulta no sea vacía antes de procesarla.

```
try:
    if not consulta or not consulta.strip(): raise ValueError("Consulta vacia")
    tag, similitud = detectar_intencion(consulta, ...)
except Exception:
    respuesta = "Ocurrio un error. Por favor intenta nuevamente."
```

6. Resultados de la Evaluación

El agente fue evaluado sobre un conjunto de 30 consultas variadas que incluyen: variaciones lingüísticas y sinónimos, errores tipográficos intencionales (inscripción, nivelasion, nibelacion), preguntas ambiguas con vocabulario compartido entre intenciones, y consultas completamente fuera del dominio. Todas las consultas del conjunto de prueba provienen del código directamente, garantizando reproducibilidad.

Métrica	Resultado
Accuracy	100.00%
F1-macro	100.00%
Total consultas evaluadas	30
Consultas correctamente clasificadas	30 / 30
Consultas con error tipográfico	3 (todas correctas)
Consultas fuera de dominio	3 (todas identificadas)

El reporte por intención generado por `classification_report` de scikit-learn muestra precision, recall y F1 de 1.00 en todas las intenciones evaluadas. Los tres casos con errores tipográficos (inscripcion, nivelasion, nibelacion) fueron clasificados correctamente gracias al stemming de Snowball, que reduce las variantes mal escritas a raíces compatibles con las del corpus de entrenamiento. Las tres consultas fuera de dominio (quien gano el mundial, como se hace una torta, recomiéndame una película) obtuvieron similitudes coseno menores a 0.05 con todos los utterances, bien por debajo del umbral de 0.25, y fueron correctamente derivadas a la intención `fuera_dominio`.

Limitaciones observadas

- El agente depende del vocabulario presente en el JSON de intenciones, con consultas muy específicas con terminología no vista en entrenamiento (ej. nombres de profesores, aulas, o eventos puntuales) podrían no superar el umbral de confianza.
- El corpus enriquece las respuestas pero no genera texto nuevo, si la respuesta predefinida es corta y el fragmento del corpus no coincide semánticamente, la respuesta puede resultar genérica.
- El agente no mantiene contexto conversacional entre turnos: cada consulta se procesa de forma independiente, por lo que puede responder preguntas de seguimiento como "y en ese

caso que hago?".

- El umbral de 0.25 fue elegido empíricamente, en una producción real sería necesario ajustarlo con un conjunto de validación más amplio que incluya consultas de usuarios reales.

7. Conclusiones

Se diseñó e implementó un agente conversacional basado exclusivamente en técnicas TF-IDF + similitud coseno, capaz de responder preguntas frecuentes sobre el proceso de admisión y nivelación de la Universidad de Guayaquil, demostró un desempeño del 100% de Accuracy y F1-macro sobre el conjunto de prueba de 30 consultas, incluyendo variaciones lingüísticas, errores tipográficos y consultas fuera del dominio.

La arquitectura modular con datos separados del código permite escalar fácilmente el agente agregando nuevas intenciones al JSON sin modificar la lógica del sistema. La integración del corpus de información académica enriquece las respuestas con datos reales sobre carreras, facultades y criterios de aprobación, superando la limitación de los chatbots que solo pueden responder con respuestas completamente fijas.

El preprocesamiento con stemming Snowball demostró ser clave para la robustez del agente frente a errores tipográficos y variaciones morfológicas del español, además el umbral de confianza de 0.25 resultó adecuado para el dominio cerrado de admisiones, evitando respuestas incorrectas a consultas fuera del alcance del sistema.

Como trabajo futuro se podría incorporar contexto conversacional entre turnos, expandir el corpus con información de cada facultad y reemplazar el stemming por lematización con spaCy para mayor precisión en el manejo del vocabulario en español.

8. Referencias

Jurafsky, D., & Martin, J. H. (2026). Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with language models (3.^a ed., borrador). <https://web.stanford.edu/~jurafsky/slp3/>

Jurafsky, D., & Martin, J. H. (2021). Vector semantics and embeddings (Cap. 6). En Speech and language processing (3.^a ed., borrador, diciembre 2021). Stanford University. https://web.stanford.edu/~jurafsky/slp3/old_dec21/6.pdf

Bird, S., Klein, E. & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media. <https://www.nltk.org/book/>

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830. <https://scikit-learn.org/>

Universidad de Guayaquil - Portal Oficial de Admision. (2026). Proceso de admision. <https://admision.ug.edu.ec/admision/>

Universidad de Guayaquil - Pagina de Nivelacion. (2026). Proceso de nivelacion. <https://admision.ug.edu.ec/nivelacion/>

Ministerio de Educacion del Ecuador. (2026). Registro Nacional de Admision a la Educacion Superior. <https://admision.educacion.gob.ec/>

Universidad de Guayaquil. (2026). Directrices del Curso de Nivelacion de Carrera 2026. <https://admision.ug.edu.ec/Docs/Reg/Normativas/Directrices%20nivelacion%202026%20CI.pdf>